Direct-Actuation Bead Tracing versus RL-Tuned Pure Pursuit for Contour and Racetrack Following

Dileep Banothu
[Department]
[Institution]
[City, Country]
[email]

N. Sanjay Varma
[Department]
[Institution]
[City, Country]
[email]

***Abstract*—The accuracy of a Pure Pursuit (PP) tracker hinges on two hand-chosen quantities, the lookahead distance and the steering gain, and a recent line of work hands their online selection to a reinforcement-learning (RL) agent. This study examines the alternative of removing the geometric law entirely: does a small network that sets the acceleration of a point-mass "bead" directly follow a reference more closely than a PP controller with tuned or learned parameters? Our pipeline first turns any raster image into an ordered reference polyline with uniform arc-length spacing, using Otsu thresholding and Zhang–Suen thinning; it then simulates a damped point mass with bounded acceleration as a Gymnasium environment; finally, it trains one actor–critic network by three separately reported procedures: expert anchoring with Dataset Aggregation (DAgger), Proximal Policy Optimization (PPO) on the environment reward, and DAgger initialization followed by PPO. Training uses 11 circuits from the F1TENTH collection plus 120 synthetic contours; checkpoints are chosen on 4 further circuits and 50 contours; and a single final test is run on 8 circuits never seen before, among them the three on which an earlier version of this work was scored. On these unseen circuits the bead trained by DAgger attains a root-mean-square tracking error of 1.07 cm, the same as the hand-written rule it imitates (1.07 cm). PPO fine-tuning worsens this to 3.31 cm, and PPO started from random weights reaches 7.40 cm after the 4×10^5 steps our budget allowed. A Fixed PP with gains tuned on the training circuits achieves 0.90 cm while completing a lap in 18.24 s, compared with 42.45 s for the slower, damping-limited bead, and it stays ahead even when its speed is lowered to the bead's. Once the baselines are tuned and the test circuits are kept separate, the earlier claim that direct actuation is more precise no longer holds. With only two training seeds and eight test circuits, no pre-registered comparison remains significant after correction for multiple testing.**

***Index Terms*—Path tracking, Pure Pursuit, imitation learning, DAgger, Proximal Policy Optimization, contour following, skeletonization, F1TENTH, autonomous racing**

I. Introduction

Every autonomous vehicle needs a component that converts a planned path into control commands while staying close to that path, whether it is straight or tightly curved, slow or fast. On small-scale racing platforms like F1TENTH [1], this job is usually given to geometric controllers, and Pure Pursuit (PP) [2], [3] is the most common of them: it is simple to analyze and light enough for embedded processors running at high rates. PP aims the vehicle at a point located a distance L_d further along the path, so its behavior is set almost entirely by this lookahead and by a steering gain g. With a small L_d the vehicle reacts quickly but weaves on straight segments; with a large one it drives smoothly but runs wide in tight corners. Rules that grow L_d with speed ease this conflict without removing it.

A natural remedy is to let a learning agent choose these values. In the approach of Elgouhary and El-Wakeel [4], the PP equations are left unchanged and a PPO [5] agent picks (L_d, g) at every step from the current speed and the upcoming curvature; the controller stays interpretable, and only two numbers pass between the agent and the existing stack. A more radical remedy discards the geometric controller and lets the network output the actuation itself. Studies seldom place these two philosophies side by side under the same simulator and the same evaluation measures.

We carry out exactly that side-by-side study. Our starting point was a simulator, developed separately, in which a point mass (the "bead") traces a contour taken from any picture. We rebuilt the RL–PP environment, reward and training procedure of [4] inside this simulator and posed a single question: given identical paths and identical scoring, is a network that drives the actuation directly more precise than a PP controller with learned parameters?

An earlier version of this study answered "yes" on three circuits. A subsequent audit of that version found that its direct-actuation policy had not been optimized with PPO at all, but regressed onto a geometric expert; that it was trained and checkpoint-selected on the circuits on which it was scored; that the expert itself, never reported, was more accurate than the learned policy; that the Fixed PP gains had not been tuned; and that the RL–PP policy had been evaluated on unnormalized observations although it was trained on normalized ones. This version corrects all five points and re-runs every experiment. **The answer is now negative.** Specifically, we contribute:

- **an image-to-reference front end** that recovers an ordered, uniformly resampled polyline from skeletons or contours, validated against the source centerlines of 23 circuits;
- **three explicitly separated training procedures** for one direct-action bead network—DAgger [6], PPO, and DAgger-initialized PPO—with model selection on validation circuits only;

- **a same-arena re-implementation of RL-tuned PP** [4], a separately trained lookahead-only variant, and two classical PP baselines tuned on training circuits, all scored under one protocol at native and at matched speed;
- **a held-out evaluation** on 8 circuits and 100 synthetic contours with pre-declared statistics, including negative results and an explicit account of threats to validity.

II. Related Work

Geometric tracking. The two best-known geometric path followers are Pure Pursuit [2] and the Stanley method [7]; Snider [3] contrasts them in detail, and the vehicle models used to design them are covered in [8], [9]. Because PP reacts strongly to the choice of lookahead, many variants adapt L_d to speed or curvature, but each still uses a hand-chosen formula whose constants rarely carry over from one track to another. In racing, the path these controllers follow is commonly produced by minimum-curvature optimization [10], [11].

Learning-based control. Surveys document the wide use of deep RL for driving [12] and for robots in general [13]. Among on-policy methods, PPO [5] combined with Generalized Advantage Estimation (GAE) [14] has become the standard starting point, although its results are sensitive to many small implementation choices [15]. Off-policy methods such as Soft Actor-Critic (SAC) [16] use samples more efficiently at some cost in stability. Networks that map observations straight to commands are flexible, yet they are harder to interpret and to transfer, as racing comparisons have shown [17]. Using RL only to adjust a classical controller, as in [4], is a compromise between these extremes.

Imitation learning. Learning by copying a demonstrator goes back to ALVINN [18]; its weakness is that small mistakes take the learner into situations the demonstrator never showed, and errors then accumulate. DAgger [6] counters this by repeatedly asking the expert to label the states the learner actually reaches and adding them to the training set, which yields a no-regret guarantee. Because such a learner can only approach the quality of its teacher, RL fine-tuning is the usual way to go further—the route we examine in this work.

Positioning. Our bead controller lies between these directions: it is free of a PP steering equation, unlike [4], and it can lean on an inexpensive geometric teacher, unlike pure end-to-end RL. We include that teacher as a baseline in every comparison, which makes it possible to tell what the network learned apart from what the teacher and the dynamics already supply.

III. Framework and Methodology

The complete system is outlined in Fig. 1. Section III-A explains how an image becomes a reference path, Sections III-B–III-E describe the bead and how it is trained, and Section III-F describes the RL–PP baseline built on the same paths.

A. Raster-to-Reference Perception

Images larger than 1200 px on their longest side are scaled down, then converted to gray levels and smoothed with a

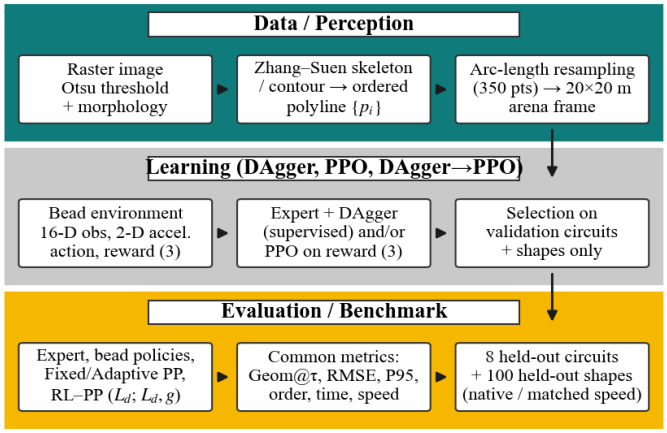


Fig. 1. System overview. Top: an input picture is turned into an ordered reference path. Middle: the bead network is trained with DAgger, with PPO, or with DAgger and then PPO, and checkpoints are picked on validation paths only. Bottom: the bead policies, the geometric expert and the Pure Pursuit family, including our rebuild of RL-tuned PP [4], are scored with the same measures on unseen circuits and shapes.

Gaussian filter. Otsu's method [19] separates the drawing from the background; whether the drawing is darker or lighter is decided from the median brightness along the image border, so either convention works. Small holes are filled by a 3×3 closing. When the drawing covers no more than 20% of the image it is treated as a thin stroke and reduced to a one-pixel skeleton by Zhang–Suen thinning [20]; larger regions are traced along their outer contours instead. Every 8-connected piece of the skeleton is converted into an ordered list of points by walking from an end pixel and, wherever the stroke branches, continuing in the direction that turns least. The points are scaled into a 20×20 m arena with the vertical axis flipped to obtain a right-handed frame, pieces shorter than 0.1 m are dropped, and the rest are resampled at equal arc-length spacing, about 350 points in total shared in proportion to length. The junctions between separate pieces are stored as breaks and skipped in every distance computation. An image from which no path can be obtained is now reported as a failure; previously the code quietly replaced it with a default circle, and this fallback is switched off for all experiments.

Because every circuit image is rasterized from a known centerline, the extraction error can be measured exactly. Over all 23 circuits the mean distance from the source centerline to the extracted reference is 0.79–1.24 cm per circuit and the maximum is 6.53 cm. This error is of the same order as the tracking errors reported below; all tracking metrics are therefore measured against the extracted reference, not against the true centerline. Fig. 2 shows the front end on four kinds of input.

B. Bead Dynamics

We model the bead as a disc of radius 0.15 m whose state is its position $\mathbf{p}_t \in \mathbb{R}^2$ and velocity $\mathbf{v}_t$. The command $\mathbf{u}_t \in [-1, 1]^2$ is scaled to an acceleration of at most $a_{\max} = 10$ m/s^2 along each axis, the motion is slowed by viscous friction $\mu =$

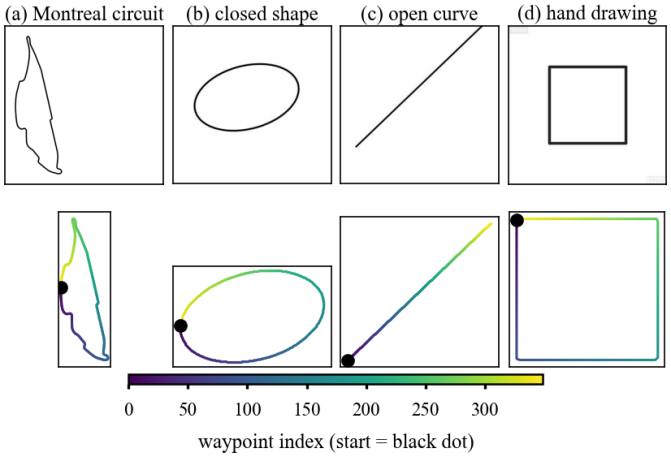


Fig. 2. Image-to-reference front end. Top: raster inputs—a rasterized test circuit, a closed shape and an open curve from the synthetic test splits, and a hand-drawn image. Bottom: the ordered references extracted from them, colored by waypoint index; the black dot marks waypoint 0.

$7\ \mathrm{s}^{-1}$ and limited to $v_{\max} = 7$ m/s, and the state is advanced every $\Delta t = 0.016$ s:

$$\begin{aligned} \mathbf{v}_{t+1} &= (\mathbf{v}_t + a_{\max}\mathbf{u}_t\Delta t)(1 - \mu\Delta t), \\ \mathbf{p}_{t+1} &= \mathbf{p}_t + \mathbf{v}_{t+1}\Delta t, \end{aligned} \tag{1}$$

where speeds above $v_{\max}$ are clipped and the bead rebounds from the arena boundary with a restitution coefficient of 0.15. Holding a full command along one axis drives (1) to the steady speed

$$v^* = a_{\max}\Delta t\,(1 - \mu\Delta t)/(\mu\Delta t) \approx 1.27\ \text{m/s}, \tag{2}$$

while a full diagonal command gives at most $\sqrt{2}\,v^* \approx 1.79$ m/s; in practice the friction, rather than $v_{\max}$, limits how fast the bead can go. An episode is truncated after $\max(3500, \lceil 1.5L/(v^*\Delta t)\rceil)$ steps for a reference of length L, so that a bead moving at v^* can always finish (the earlier fixed cap of 3500 steps could not cover the longest circuits), or after 400 consecutive steps without progress. The same cap is applied to the PP vehicle.

C. Observation and Action Spaces

The policy receives a 16-element vector $\mathbf{o}_t$ whose entries lie in $[-1, 1]$, expressed relative to the bead and divided by the arena half-width $N = 10$ m. Its entries are: the vectors to the current and following waypoints (4 values); the scaled position (2) and velocity (2); the distance to the closest reference point searched within ± 20 indices (1); a flag telling whether the bead is on the path (1); the cosines of the direction of motion (2); a signed turning-angle measure of the local curvature (1); the unit vector to the waypoint five steps ahead (2); and the fraction of the path completed (1). The output is the 2-D acceleration command $\mathbf{u}_t$. The waypoint counter w_t can move forward by at most four indices per step and only to points within the coverage radius $\rho = 0.18$ m, so the bead cannot skip ahead where the path doubles back near itself.

TABLE I
REWARD WEIGHTS OF THE BEAD ENVIRONMENT

Term	Symbol	Weight	Term	Symbol	Weight
Approach	w_a	0.10	Off-curve	w_o	0.10
Waypoint	w_w	0.40	Backtrack	w_b	0.05
Centering	w_e	0.25	Wall contact	w_c	0.50
Tangency	w_τ	0.15	Action jerk	w_j	0.005
Tracking error	w_ℓ	0.05	Step cost	w_s	0.001

D. Reward Design

Writing e_t for the distance to the closest reference point, d_t for the distance to waypoint w_t, n_t for the number of waypoints passed in the current step, and $\hat{\mathbf{r}}_t$ for the unit tangent of the path, the reward at each step is

$$\begin{aligned} r_t = {} & w_a\,\mathrm{clip}\big((d_{t-1} - d_t)/\rho, -1, 1\big) + w_w n_t \\ & + w_e \exp[-(e_t/\sigma)^2] \\ & + w_\tau\,\mathrm{clip}(\mathbf{v}_t \cdot \hat{\mathbf{r}}_t/v_{\max}, -1, 1) - w_\ell \min(e_t/\rho_s, 3) \\ & - w_o \mathbb{1}[e_t > \rho] - w_b \mathbb{1}_{\text{back}} - w_c \mathbb{1}_{\text{wall}} \\ & - w_j \min(\|\mathbf{u}_t - \mathbf{u}_{t-1}\|, 1) - w_s, \end{aligned} \tag{3}$$

in which $\mathbb{1}_{\text{back}}$ flags a step backward along the path, $\mathbb{1}_{\text{wall}}$ flags contact with the boundary, $\sigma = 0.08$ m is the width of the Gaussian bonus for staying centered, and $\rho_s = 0.10$ m is the tight tolerance. At the end of an episode the agent gains $+50$ if the waypoint counter reached 99% of the path, $+15$ if the episode was cut off after passing 80%, and -5 in every other case. The weights, kept from the earlier version, appear in Table I. In this version the reward is the training signal of the PPO variants (Section III-E); the DAgger variant does not use it.

E. Policy Architecture and Learning

We use the actor–critic of Stable-Baselines3 [21], giving the actor and the critic their own two-layer networks of 256 tanh units and using a diagonal Gaussian over actions whose spread does not depend on the state. At test time the mean action is applied, with no sampling. The hand-written teacher simply steers toward the waypoint after the current one,

$$\mathbf{u}_t^* = (\mathbf{p}[w_t + 1] - \mathbf{p}_t)/\|\mathbf{p}[w_t + 1] - \mathbf{p}_t\|. \tag{4}$$

The same network is trained in three ways, which we name by the procedure that actually updates it.

Bead-DAgger first fits the actor to one teacher-driven episode on each training path and then improves it with DAgger [6]: the network drives, the teacher supplies the correct action for every state reached, and the growing dataset is fitted with a squared-error loss on the actor branch (Algorithm 1). Neither the critic nor the reward plays any part.

Bead-PPO optimizes the network from a random initialization with the clipped surrogate objective of PPO [5],

$$\mathcal{L}(\theta) = \mathbb{E}_t\big[\min(\varrho_t \hat{A}_t, \mathrm{clip}(\varrho_t, 1 \pm \epsilon)\hat{A}_t)\big] - c_v\mathcal{L}^V + c_e\mathcal{H}, \tag{5}$$

where ϱ_t is the probability ratio between the new and the rollout policy, $\hat{A}_t$ is the GAE [14] advantage of the reward

Algorithm 1 Expert anchoring and DAgger refinement (Bead-DAgger)

Input: training references $\mathcal{T}$, validation references $\mathcal{V}$, expert π^* (4), network π_θ, mixing schedule $\beta = (0.4, 0.2, 0.1, 0.05, 0, 0, 0, 0)$
$\mathcal{D} \leftarrow$ one expert episode per $\tau \in \mathcal{T}$; fit actor on $\mathcal{D}$ (12 epochs, lr 2×10^{-4})
for round $k = 1, \ldots, 8$ **do**
  for each $\tau \in \mathcal{T}$, roll out one episode executing π^* w.p. β_k, else π_θ; label every state with $\pi^*(o)$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o, \pi^*(o))\}$; refit actor (8 epochs, lr 1.5×10^{-4} for $k \leq 3$, else 7×10^{-5})
  evaluate the validation score on $\mathcal{V}$; keep the checkpoint if it improves
end for
return best validation checkpoint

TABLE II
LEARNING HYPERPARAMETERS

Quantity	Bead (PPO variants)	RL–PP (re-impl. of [4])
Policy / value MLP	$[256, 256]$ tanh	SB3 default $[64, 64]$
Rollout n_{steps} / batch	2048×4 envs / 128	2048 / 256
Epochs per update	10	5
γ / GAE λ	0.99 / 0.95	0.99 / 0.98
Clip ϵ / target KL	0.2 / –	0.2 / 0.015
Entropy / value coeff.	0.001 / 0.5	0.02 / 0.6
Grad-norm clip	0.5	0.7
Learning rate	3×10^{-4}	2.4×10^{-4}, linear decay
Initial $\log \sigma$	-0.5 (pilot-selected)	0 (default)
Normalization	–	VecNormalize (obs, return)‡
Environment steps	4×10^5 per run	10^5 per run
Training seeds	0, 1	0, 1

‡Statistics saved with each checkpoint and applied at evaluation.

(3), $\mathcal{L}^V$ the value loss and $\mathcal{H}$ the policy entropy. Time-limit truncations are bootstrapped with the critic. The hyperparameters (Table II) are those declared for the network in the earlier version, where they were never exercised; the only addition, the initial log standard deviation, was chosen from $\{-1, -0.5, 0\}$ by a validation-only pilot with a seed that is not reported.

Bead-DAgger→PPO copies the actor of the Bead-DAgger checkpoint of the same seed, trains the critic for 5×10^4 steps with the actor frozen, and then continues with (5) under the same settings and the same step budget as Bead-PPO.

All three variants sample training references from the training split, with circuits making up about half of all episodes, and keep the checkpoint with the best validation score: the mean Geom@0.10 minus the mean RMSE in meters, averaged with equal weight over validation circuits and validation shapes. No test reference is used before the final evaluation.

F. Re-implementation of RL-Tuned Pure Pursuit

To give both controllers the same paths and the same scoring, we rebuilt the method of [4] in our arena. The car is simulated with a kinematic bicycle model [8] of wheelbase $W = 0.33$ m. PP takes the sideways offset y' of the lookahead point in the car's own frame, converts it into a desired curvature, and turns that curvature into a steering angle:

$$\gamma = g \cdot 2y'/L_d^2, \qquad \dot{\theta} = (v/W)\tan\delta, \\ \delta = \text{clip}(\arctan(W\gamma), \pm 1.10 \text{ rad}), \tag{6}$$

As in [4], the agent sees the speed, the curvature magnitudes $\kappa_{0,1,2}$ at 0, 5 and 12 waypoints ahead, and their difference $\Delta\kappa = \kappa_1 - \kappa_0$; we add the signed sideways error and the sine and cosine of the heading error, for eight inputs in total. Its two outputs are mapped to $L_d \in [0.20, 1.20]$ m and $g \in [0.60, 1.40]$ and then low-pass filtered as in [4], with $\beta_L = \beta_g = 0.2$:

$$\tilde{L}_{t+1} = \beta_L L_{t+1} + (1 - \beta_L)\tilde{L}_t, \\ \tilde{g}_{t+1} = \beta_g g_{t+1} + (1 - \beta_g)\tilde{g}_t. \tag{7}$$

Our arena is several times smaller than a real F1TENTH track, so the original parameter ranges and teacher constants had to be scaled down. The teacher's targets are $L^* = \text{clip}(0.65 - 0.18\kappa_{\text{max}} + 0.04v,\ 0.26,\ 0.85)$ and $g^* = \text{clip}(0.96 + 0.14\kappa_{\text{max}},\ 0.60,\ 1.40)$. Speed follows a friction-limited target $v_{\text{ref}} = \text{clip}(4.2/\sqrt{1 + 2.5|\kappa|}, 1.8, 5.0)$ m/s, smoothed by a backward braking pass (3.5 m/s^2) and a forward acceleration pass (2.5 m/s^2), and the car tracks it with $a = 3(v_{\text{ref}} - v)$. The reward contains the terms of [4]—closeness to the teacher, penalties on abrupt parameter changes, a curvature-dependent lookahead term, and terms for progress, speed, collisions and stalling, clipped to $[-30, 100]$—plus three additions: a Gaussian corridor bonus $3.5\exp[-\frac{1}{2}(e/0.035)^2]$, a quadratic penalty for errors above 0.07 m, and an alignment bonus $0.8\cos e_\theta$. The joint and the L_d-only RL–PP agents are now trained independently, whereas the earlier version took the L_d-only controller from the joint network's output.

Two classical controllers complete the set: Fixed PP with constant (L_d, g) and Adaptive PP with $L_d = \text{clip}(a + bv, 0.20, 1.20)$, $g = 1.05$. Both were grid-searched on the training references (16×6 and 9×6 grids) with the same selection score as the learned controllers. Fixed PP selects $(L_d, g) = (0.25 \text{ m}, 0.90)$, and Adaptive PP selects $(a, b) = (0.10, 0.00)$, i.e. a constant lookahead at the 0.20 m lower bound. Both optima lie on a boundary of the search space. The original gains $(0.55 \text{ m}, 1.05)$ and schedule $(0.35, 0.25)$ are reported alongside.

IV. EXPERIMENTAL SETUP

A. References and Datasets

We rasterized all 23 circuits of the public F1TENTH racetrack collection[1] with the same 1200-px, 7-px-stroke procedure as before (the three images of the earlier version are reproduced bit-identically) and passed them through the front end of Section III-A. Hockenheim, Montreal, and Yas Marina, the circuits of the earlier version, were assigned to the test split; the remaining 20 were split by a seeded permutation, fixed before any training, into 5 further test, 4 validation, and 11 training circuits (Fig. 3). No circuit appears in more than one split. Training additionally uses 60 closed shapes and 60 open

[1]https://github.com/f1tenth/f1tenth_racetracks, commit b95c4ef.

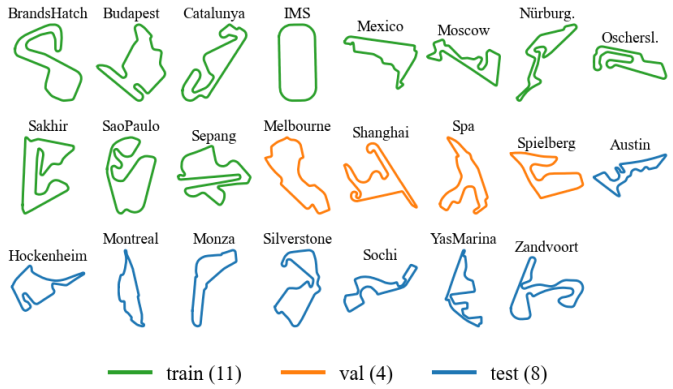


Fig. 3. Extracted references of the 23 F1TENTH circuits, colored by split. The test split contains the three circuits of the earlier version (Hockenheim, Montreal, Yas Marina) and five further circuits; no test circuit is used for training, checkpoint selection, or baseline tuning.

curves from the training splits of two synthetic datasets of 1000 images each (polygons, ellipses, and rounded quadrilaterals; arcs, lines, and parabolas); 25 of each validation split are used for selection and the first 50 of each test split for testing. Synthetic splits are i.i.d. draws from the same generator, so they test in-distribution generalization; the circuits test transfer to unseen layouts. In total 493 references were passed through the front end—23 circuits, 120 training, 50 validation and all 300 test-split synthetic contours—and all were extracted without failure; of the 300 synthetic test references, the first 50 per set (100) were evaluated.

B. Metrics

For a reference $P = (\mathbf{p}_0, \dots, \mathbf{p}_{N-1})$ and a driven trajectory $X = (\mathbf{x}_0, \dots, \mathbf{x}_T)$, geometric coverage with tolerance τ is the share of reference points lying within τ of the trajectory polyline:

$$\text{Geom@}\tau = \frac{1}{N} \sum_{\mathbf{p} \in P} \mathbb{1}[\text{dist}(\mathbf{p}, X) \le \tau]. \tag{8}$$

In the other direction, the distance $E = \text{dist}(\mathbf{x}, P)$ of each trajectory sample from the reference gives the root-mean-square tracking error (RMSE), its 95th percentile (P95) and its maximum. Geom@τ does not care about the order of visits. Ordered coverage K/N counts the largest set of reference points $i_1 < \cdots < i_K$ that are each visited, $\|\mathbf{x}_{t_k} - \mathbf{p}_{i_k}\| \le \rho$, at non-decreasing times $t_1 \le \cdots \le t_K$; dynamic programming evaluates it exactly, and it punishes skipped or reversed sections while still crediting the remainder of a lap after a single excursion. We count a run as completed when its ordered coverage is 0.95 or more. A single piece of code computes every measure from the saved trajectories of all controllers, whereas the earlier version scored the bead and the car with two different progress rules. Lap time equals the number of steps times Δt.

C. Protocol

Every controller was executed for five episodes per test circuit with seeds $\{50000+137k\}$, $k = 0, \dots, 4$, which perturb the initial position by up to ± 5 cm, and for one episode per test shape. Policies were run deterministically. Each learned controller was trained with two seeds, a number limited by our compute budget; Table III reports the mean over seeds with the range across seeds in brackets. Track A runs each controller at its native pace; Track B caps the PP reference speed at $v_{\text{match}} = 1.256$ m/s, the mean speed of the expert on the training circuits, with Fixed and Adaptive PP re-tuned and RL–PP retrained at that speed. Pre-declared comparisons are paired over the 8 test circuits: an exact two-sided Wilcoxon signed-rank test with Holm correction over 14 tests, and the mean paired difference with a 95% bootstrap interval that resamples circuits and training seeds. With $n = 8$ the smallest attainable two-sided p is 0.0078. Table III lists 15 controllers; counting each training seed separately they form 21 evaluated instances: 9 controllers with a single instance (the expert, the six classical PP configurations, and the two published checkpoints) and 6 learned controllers with two seeds each (Bead-DAgger, Bead-PPO, Bead-DAgger→PPO, RL–PP joint, RL–PP L_d only, RL–PP joint at v_{match}). Each instance ran $8 \times 5 = 40$ circuit episodes and 100 contour episodes, giving $21 \times 140 = 2940$ test episodes, executed on a CPU-only workstation using PyTorch 2.11, Stable-Baselines3 2.8 [21], and Gymnasium 1.2 [22]. For verification, every metric was recomputed from the stored trajectories, RMSE and Geom@0.10 additionally with an independent implementation, twelve controller instances were re-executed from their saved checkpoints with bit-identical trajectories, and all tables were regenerated from the raw records; the scripts are part of the released code.

V. Results and Discussion

This section starts with accuracy on the unseen circuits, then looks at the effect of PPO on the bead (Section V-B), at the balance between precision and speed (Section V-C) and at the synthetic contours (Section V-D). It continues with the schedule learned by RL–PP (Sections V-E–V-F) and ends with the threats to validity.

A. Circuit-Level Tracking Accuracy

Table III and Fig. 6 consolidate the held-out circuit results. Among the bead controllers, **Bead-DAgger matches its geometric expert almost exactly** (RMSE 1.07 vs. 1.07 cm, Geom@0.10 99.1% vs. 99.2%), with negligible difference between its two seeds. Both PPO variants are worse: Bead-DAgger→PPO reaches 3.31 cm and Bead-PPO from scratch 7.40 cm, the latter covering only 78.90% of the reference within 0.10 m and completing 25% of the test runs. The published checkpoint of the earlier version reaches 4.38 cm on these circuits; on the three circuits it was trained on, it reproduces the earlier figures (Table VI).

The PP family is at least as accurate. Tuned Fixed PP reaches 0.90 cm and tuned Adaptive PP 0.63 cm, both below the expert and every bead policy, while traversing a

TABLE III

TRACKING PERFORMANCE ON THE EIGHT HELD-OUT TEST CIRCUITS (MEAN OVER CIRCUITS, FIVE EPISODES EACH; LEARNED CONTROLLERS: MEAN OVER TWO TRAINING SEEDS WITH THE RANGE ACROSS SEEDS IN BRACKETS; BEST ACCURACY PER COLUMN IN BOLD; TIME AND SPEED NOT RANKED)

Controller	Geom@0.05 [%]↑	Geom@0.10 [%]↑	RMSE [cm]↓	P95 [cm]↓	Max [cm]↓	Ordered [%]↑	Completed [%]	Time [s]	Speed [m/s]
Bead plant (holonomic point mass, damping-limited)									
Geometric expert (4)	**98.6**	**99.2**	1.07	2.55	5.02	**99.1**	100	42.5	1.25
Bead-DAgger	98.4	99.1 [99.1, 99.1]	1.07 [1.06, 1.07]	2.48	5.46	**99.1**	100	42.4	1.25
Bead-DAgger→PPO	85.4	98.0 [97.0, 99.0]	3.31 [3.06, 3.57]	6.43	11.46	**99.1**	100	40.8	1.29
Bead-PPO (scratch)	60.4	78.9 [78.8, 79.0]	7.40 [7.16, 7.65]	16.27	23.67	93.4	25	50.0	1.01
Published Bead ckpt.*	77.0	93.9	4.38	9.26	16.30	98.9	100	42.5	1.23
Bicycle plant, native speed profile (Track A)									
Fixed PP (orig. gains)	84.8	94.3	3.89	8.71	18.98	97.8	100	17.8	2.90
Fixed PP (tuned)	97.4	98.2	0.90	2.09	4.94	98.1	100	18.2	2.87
Adaptive PP (orig.)	53.1	71.8	11.94	26.83	46.19	84.9	12	16.7	2.98
Adaptive PP (tuned)	97.9	98.1	0.63	1.40	4.11	98.1	100	18.2	2.87
RL–PP (joint L_d, g) [4]	97.5	98.2 [98.1, 98.2]	1.03 [0.99, 1.07]	2.23	5.70	98.1	100	18.2	2.87
RL–PP (L_d only)	97.5	98.2 [98.2, 98.2]	0.98 [0.98, 0.98]	2.08	5.35	98.1	100	18.2	2.87
Published RL–PP ckpt.†	76.2	92.4	5.00	10.78	22.08	97.3	100	17.8	2.90
Bicycle plant, speed capped at $v_{\text{match}} = 1.256$ *m/s (Track B)*									
Fixed PP (tuned)	97.4	98.1	0.77	1.80	4.89	97.9	100	41.6	1.26
Adaptive PP (tuned)	97.8	98.1	**0.54**	**1.17**	**4.04**	97.9	100	41.7	1.26
RL–PP (joint L_d, g)	97.5	98.1 [98.1, 98.1]	0.88 [0.66, 1.10]	1.94	5.29	97.9	100	41.6	1.26

*Checkpoint of the earlier version; trained and selected on Hockenheim, Montreal and Yas Marina, which are three of the eight test circuits here.

†Checkpoint of the earlier version (trained on one hand-drawn contour); evaluated on raw observations as in that version, because its normalization statistics were not saved.

lap in 18.24 s instead of 42.45 s. Retrained RL–PP reaches 1.03 cm. The gap between these values and the earlier version is explained by two corrections: with the original gains Fixed PP reaches 3.89 cm, and the published RL–PP checkpoint, evaluated on unnormalized observations, 5.00 cm. Geom@0.10 is about one point lower for every PP controller than for the expert (98.2% vs. 99.2%). On every test circuit the only reference points left beyond 0.10 m by tuned Fixed PP and by the expert are the last waypoints before the start/finish line: the PP lap-completion rule ends an episode six to seven waypoints before loop closure and the bead's 99% rule three. Geom@τ differences of this size are therefore an artifact of episode termination, not of tracking. Ordered coverage is at least 97.9% and every PP run is completed.

Fig. 4 shows complete laps on the three circuits of the earlier version, and Fig. 5 the distribution of the lateral error over all test circuits: the expert, Bead-DAgger, RL–PP and both tuned PP laws keep at least 96.9% of the trajectory samples within 3 cm of the reference, against 65.8% for Bead-DAgger→PPO and 40.5% for Bead-PPO.

Fig. 7 shows where the bead policies fail. At the Montreal corners where Bead-DAgger→PPO deviates most, it rounds the corner exit, whereas the expert, Bead-DAgger, tuned Fixed PP and RL–PP follow nearly coincident paths; Bead-PPO from scratch cuts the corner.

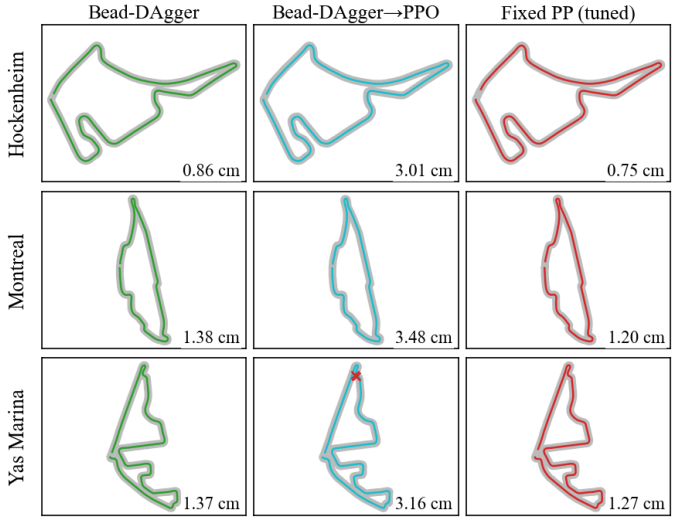


Fig. 4. Full-lap trajectories on the three circuits of the earlier version, now held out (first episode, training seed 0), over the reference (grey). Red crosses: reference points left farther than 0.10 m, excluding the start/finish closure. Inset: tracking RMSE of the lap.

B. What PPO Does to the Bead Policy

The PPO variants are genuinely optimized on the reward (3): actor, critic and action noise are all updated (verified by unit tests, including the exact transfer of the DAgger actor and its freezing during the critic warm-up, where the approximate KL divergence stayed below 10^{-9}). The training diagnostics are unremarkable: the final approximate KL per update lies between 0.005 and 0.016, and the critic's explained variance reaches 0.71–0.92. The validation error of Bead-PPO falls monotonically for both seeds, from about 10.8 to 8.0 cm over 4×10^5 steps (Fig. 8). The budget was too small for convergence; the selected configuration was still improving when its validation-only pilot was stopped at 7.5×10^5 steps. PPO fine-tuning of Bead-DAgger did not improve it: the validation error rose from 0.87 cm to 3.5–4.2 cm at the first evaluation after the critic warm-up and stayed between 3.0 and 4.3 cm.

On the test circuits the deterministic policies rank the same by reward and by accuracy: the expert earns the highest cumulative reward (715), Bead-DAgger nearly the same (714), Bead-DAgger→PPO less (624), and Bead-PPO least (517). PPO did, however, optimize a stochastic policy, so we ran

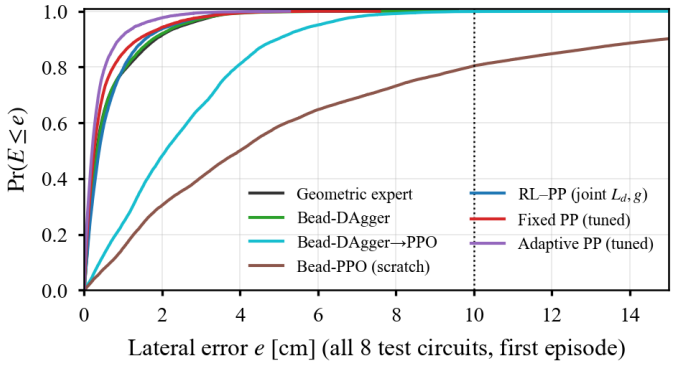


Fig. 5. Cumulative share of trajectory samples below a given distance from the reference, pooled over the eight test circuits (first episode, training seed 0 where applicable). Dotted line: 0.10 m.

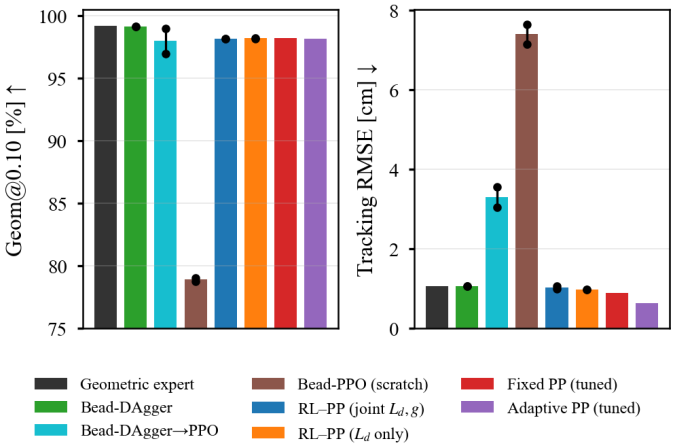


Fig. 6. Mean Geom@0.10 (left; larger is better) and RMSE (right; smaller is better) over the eight test circuits. Each dot is one training seed of a learned controller.

a validation-only diagnostic: each policy was executed on the four validation circuits with and without Gaussian action noise of the standard deviation used by PPO (three noise seeds, two training seeds). Averaged over seeds, deterministic Bead-DAgger earns 746 (RMSE 0.87 cm, 48 s); with PPO's initial noise it earns 766 (2.84 cm, 60 s). After fine-tuning, the policy earns 632 with its own noise and 604 without (3.90 cm, 45 s). Two conclusions follow. First, **within the budget, PPO fine-tuning lowered rather than raised the return of the very stochastic policy it optimizes** on held-out circuits, so the loss of accuracy is not a mismatch between stochastic training and deterministic evaluation; the episode rewards logged during training rose, but they mix references of very different length and are not comparable across iterations. Second, **the reward (3) is not monotone in accuracy**: the noisy DAgger policy is three times less accurate and slower, yet earns more, because the centering bonus is paid at every step and a slower traversal collects it for longer. Even a fully converged PPO policy would therefore not necessarily maximize tracking accuracy. Which factor—learning rate, the mixture of circuits and short contours in training, critic error, or the budget—caused the

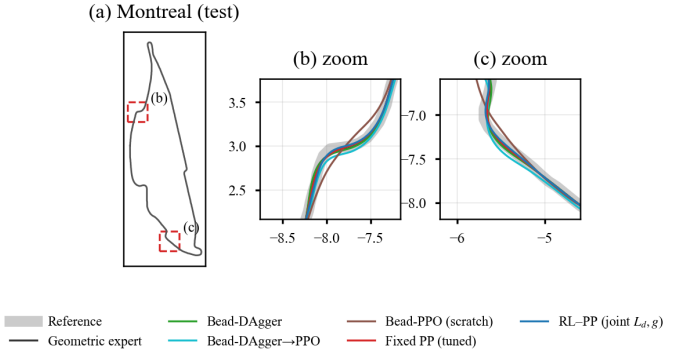


Fig. 7. Qualitative comparison on the held-out Montreal circuit (first episode, training seed 0). The zoom windows are centered, automatically, on the two reference points that Bead-DAgger→PPO misses by the largest distance, excluding the start/finish region.

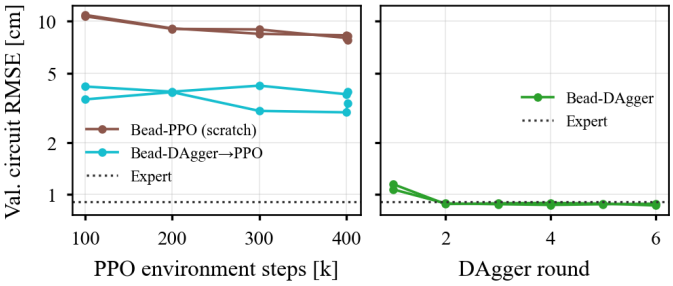


Fig. 8. Validation-circuit RMSE during training (two seeds each). Left: PPO from scratch and DAgger-initialized PPO. Right: DAgger rounds (stopped after six of eight rounds; the score was flat from round 2). Dotted: geometric expert on the same validation circuits.

fine-tuning to degrade the policy was not isolated. We report this as a negative result: under our settings, reward-driven fine-tuning did not refine the imitation policy, and the expert is the accuracy ceiling of every bead variant evaluated here.

C. The Precision–Pace Trade-off

A bead lap lasts roughly 2.3 times as long as a lap of the PP car at its own speed (Fig. 9). This gap comes from the dynamics: according to (2), friction holds the bead near 1.3 m/s per axis, while the car follows a speed plan that reaches 5 m/s on straights. The earlier version already warned that accuracy cannot be compared fairly at different speeds, and Track B addresses this. When the car's target speed is limited to v_{match}, its lap time becomes close to the bead's (Table III) and its error drops a little further: 0.77 cm for re-tuned Fixed PP, 0.54 cm for Adaptive PP and 0.88 cm for retrained RL–PP. **At matched speed, the bicycle-model PP controllers track the extracted references more closely than every bead controller, including the expert.**

D. Generalization to Held-Out Contours

Table IV reports the 100 synthetic test contours. Every controller except Bead-PPO from scratch and the two published checkpoints exceeds 98% Geom@0.10 on both sets; the expert, Bead-DAgger and tuned Fixed PP all lie between 1.0 and

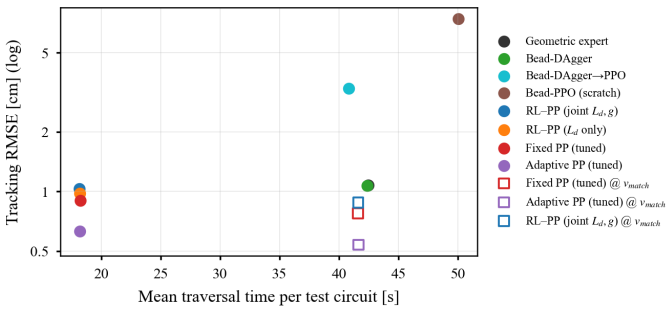


Fig. 9. Precision versus pace on the test circuits. Filled circles: native speed (Track A). Open squares: PP controllers with the reference speed capped at v_{match} (Track B).

TABLE IV
RESULTS ON HELD-OUT SYNTHETIC TEST CONTOURS (MEAN ± STANDARD DEVIATION ACROSS CONTOURS; BEST PER SET IN BOLD)

Set / Controller	Geom@0.10 [%]	RMSE [cm]	Ordered [%]	Time [s]
Closed shapes (50)				
Geometric expert (4)	**99.1** ± 0.3	1.12	**99.1**	24.4
Bead-DAgger	**99.1** ± 0.4	1.17	**99.1**	24.4
Bead-DAgger→PPO	98.9 ± 0.4	3.03	**99.1**	23.4
Bead-PPO (scratch)	96.5 ± 2.0	3.39	99.0	29.3
Fixed PP (tuned)	98.8 ± 0.5	**1.06**	98.7	10.7
RL–PP (joint L_d, g) [4]	98.8 ± 0.5	1.22	98.7	10.7
Published Bead ckpt.*	96.2 ± 2.8	3.90	99.0	25.1
Published RL–PP ckpt.†	96.8 ± 2.1	2.96	98.8	10.7
Open curves (50)				
Geometric expert (4)	98.6 ± 0.1	**1.06**	99.1	15.5
Bead-DAgger	98.6 ± 0.1	1.09	99.1	15.5
Bead-DAgger→PPO	98.6 ± 0.1	2.41	99.1	15.2
Bead-PPO (scratch)	98.6 ± 0.1	1.81	99.1	18.2
Fixed PP (tuned)	98.6 ± 0.3	1.09	99.0	8.5
RL–PP (joint L_d, g) [4]	**98.7** ± 0.4	1.25	99.1	8.5
Published Bead ckpt.*	98.4 ± 0.6	2.58	99.1	15.9
Published RL–PP ckpt.†	98.6 ± 1.0	1.86	**99.5**	8.5

1.2 cm RMSE. On short, simple contours the metric ceiling is reached by every adequate controller, and the ranking among them is within a few millimeters.

E. Interpretability of the Learned PP Schedule

Fig. 10 shows how the retrained joint RL–PP agent sets its parameters on Montreal. The lookahead uses the whole allowed interval, 0.20–1.20 m, and falls as the upcoming curvature rises ($r = -0.50$), matching the trend described in [4]: the agent looks closer before corners and farther on straights. Unlike in the earlier version, where the gain stayed within $0.96 \le g \le 1.04$, the steering gain now varies between 0.84 and 1.22 (standard deviation 0.11) and increases in corners ($r = +0.48$), as the teacher prescribes. The difference coincides with the correction of the observation normalization.

F. Ablation: Joint versus Lookahead-Only Tuning

This time the L_d-only agent was trained on its own, so the ablation concerns training and not only inference as before. Dropping the gain output still makes no difference: the L_d-only agent is very slightly more accurate than the joint one (Table V), and neither beats tuned Fixed PP. In our small arena and with our budget, learned PP parameters therefore offered

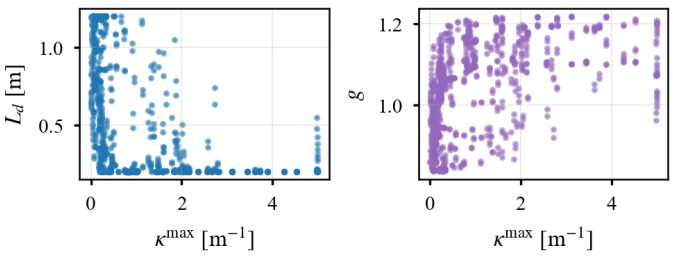


Fig. 10. Learned joint RL–PP schedule on the held-out Montreal circuit (training seed 0): L_d (left) and g (right) versus previewed curvature $\kappa_{\max}$.

no advantage over good constant ones; this says nothing against the hardware results of [4].

G. Statistical Evidence

Table V lists the pre-declared comparisons. Several effects are consistent across all eight circuits—DAgger→PPO is less accurate than DAgger, than the expert, and than tuned Fixed PP at either speed on every circuit—and their bootstrap intervals exclude zero. Their unadjusted $p = 0.0078$ is, however, the smallest value attainable with eight circuits, and after Holm correction over 14 tests **none reaches 0.05**. We therefore describe these differences as consistent but not statistically established. The two-seed intervals reflect only a minimal sample of training variability.

H. Threats to Validity

We revisit the limitations of the earlier version and add new ones. (i) Training exposure and (ii) model selection: resolved; no test circuit is used for training, checkpoint selection, or tuning. The test circuits were, however, known to the authors when the reward and the network were designed in the earlier version. (iii) The dynamics differ: partly addressed; Track B matches speed, but the bead remains holonomic and the PP vehicle non-holonomic. (iv) Actor-only training: resolved; PPO now optimizes actor and critic on the reward, but with 4×10^5 steps it is under-trained, and its fine-tuning hyper-parameters were not tuned. (v) Single models: partly addressed; two training seeds instead of one, which bounds rather than estimates seed variance. (vi) Scale and hardware: unchanged; all results are in simulation at arena scale, no physical experiments were performed, and the RL–PP ranges and teacher were rescaled from [4]. New: (vii) reference-extraction error (up to 6.53 cm) is of the same order as the reported tracking errors; (viii) the tuned PP optima lie on a boundary of their grids, so the PP baselines may be further improvable; (ix) DAgger was stopped after six of eight rounds and only 50 of 150 synthetic test contours per set were used, both for compute reasons; (x) synthetic test contours share their generator with the training contours; (xi) the reward (3) pays a per-step centering bonus and therefore favors slower traversal (Section V-B), so it is an imperfect proxy for the accuracy metrics; (xii) the PPO conclusions hold for one hyper-parameter configuration, 4×10^5 steps and two seeds, and do not establish that PPO cannot reach expert-level accuracy on this task.

TABLE V
PRE-DECLARED PAIRED COMPARISONS ON THE EIGHT TEST CIRCUITS (DIFFERENCE A−B; RMSE LOWER AND GEOM@0.10 HIGHER IS BETTER FOR A)

Comparison (A vs B)	Metric	Mean diff.	95% bootstrap CI	A better on	Wilcoxon p	Holm p
DAgger→PPO vs DAgger	RMSE	+2.24 cm	[+1.79, +2.68]	0/8	0.0078	0.109
DAgger→PPO vs DAgger	Geom@0.10	-1.15 pp	[-2.91, -0.04]	0/8	0.0078	0.109
PPO (scratch) vs DAgger→PPO	RMSE	+4.09 cm	[+2.89, +5.44]	0/8	0.0078	0.109
PPO (scratch) vs DAgger→PPO	Geom@0.10	-19.06 pp	[-23.97, -13.98]	0/8	0.0078	0.109
DAgger→PPO vs Expert	RMSE	+2.24 cm	[+1.80, +2.65]	0/8	0.0078	0.109
DAgger→PPO vs Expert	Geom@0.10	-1.19 pp	[-2.89, -0.07]	0/8	0.0078	0.109
DAgger→PPO vs Fixed PP	RMSE	+2.41 cm	[+1.95, +2.83]	0/8	0.0078	0.109
DAgger→PPO vs Fixed PP	Geom@0.10	-0.23 pp	[-1.95, +0.91]	4/8	0.6406	1.000
DAgger→PPO vs Fixed PP @v_{match}	RMSE	+2.54 cm	[+2.09, +2.95]	0/8	0.0078	0.109
DAgger→PPO vs Fixed PP @v_{match}	Geom@0.10	-0.17 pp	[-1.97, +0.99]	4/8	0.7422	1.000
RL–PP joint vs RL–PP L_d	RMSE	+0.05 cm	[-0.01, +0.13]	2/8	0.0547	0.328
RL–PP joint vs RL–PP L_d	Geom@0.10	-0.03 pp	[-0.09, +0.01]	1/8	0.5000	1.000
RL–PP joint vs Fixed PP	RMSE	+0.13 cm	[-0.06, +0.31]	3/8	0.3125	1.000
RL–PP joint vs Fixed PP	Geom@0.10	-0.05 pp	[-0.11, +0.00]	0/8	0.2500	1.000

TABLE VI
GEOM@0.10 [%] / RMSE [CM] ON THE THREE CIRCUITS OF THE EARLIER VERSION (NOW HELD OUT; BEST PER COLUMN IN BOLD)

	Hockenheim		Montreal		Yas Marina	
Controller	G@0.10	RMSE	G@0.10	RMSE	G@0.10	RMSE
Geometric expert (4)	**99.1**	0.94	**99.3**	1.24	**99.1**	1.36
Bead-DAgger	**99.1**	0.87	**99.3**	1.37	**99.1**	1.40
Bead-DAgger→PPO	98.9	3.30	98.1	3.70	96.6	3.56
Published Bead ckpt.*	**99.1**	2.45	96.1	3.45	95.6	4.05
Fixed PP (orig. gains)	95.8	3.62	90.7	5.74	90.7	5.22
Fixed PP (tuned)	98.1	**0.75**	98.4	1.20	98.1	1.28
RL–PP (joint L_d, g) [4]	98.1	0.93	98.4	**1.19**	98.0	**1.03**
Published RL–PP ckpt.†	94.6	5.16	90.3	6.42	90.0	6.24

VI. CONCLUSION

This work tested, under shared paths and shared measures, whether a network that commands actuation directly can outtrack a PP controller with learned parameters. The direct controller was a point-mass bead fed by an image-to-path front end and trained with DAgger, with PPO, or with both in sequence; the comparison set consisted of our rebuild of RL-tuned PP [4] and two classical PP controllers.

With test circuits held out and all baselines tuned on training data, the answer is no. The DAgger-trained bead reproduces its geometric expert to within a millimeter, so the direct-actuation accuracy is the expert's, not a product of learning. PPO, applied to the same reward, did not improve on that expert within our budget, either from scratch or as fine-tuning; a validation diagnostic shows that fine-tuning lowered even the return it optimizes, and that the reward itself rewards slow traversal more than accuracy. A Fixed PP with tuned gains tracks more closely at more than twice the pace, and more closely still at matched speed. Retrained RL–PP reproduces the qualitative lookahead behavior of [4], and now also a curvature-dependent gain, but is not more accurate than tuned Fixed PP. These differences are consistent across circuits but not statistically established with two seeds and eight circuits.

These limitations suggest the next steps: (a) running PPO much longer, with more seeds and tuned fine-tuning settings, to find out whether reward-based learning can surpass the teacher; (b) redesigning the reward so that its shaping terms do not accumulate with episode length and centimeter-scale errors outweigh progress; (c) testing a pursuit-style law on the bead itself, to separate the effect of the dynamics from that of the controller; (d) making path extraction more accurate than the tracking errors being measured; and (e) deploying the bead policy on a real F1TENTH car as a path generator for a lower-level steering controller.

ACKNOWLEDGMENT

We are grateful to the F1TENTH community for publishing its track data, and to Elgouhary and El-Wakeel [4], whose detailed description of their method allowed us to rebuild it in our simulator.

REFERENCES

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," in *Proc. NeurIPS 2019 Competition and Demonstration Track*, PMLR vol. 123, 2020, pp. 77–89.

[2] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon Univ., Robotics Inst., Tech. Rep. CMU-RI-TR-92-01, 1992.

[3] J. M. Snider, "Automatic steering methods for autonomous automobile path tracking," Carnegie Mellon Univ., Robotics Inst., Tech. Rep. CMU-RI-TR-09-08, 2009.

[4] M. Elgouhary and A. S. El-Wakeel, "Learning to tune pure pursuit in autonomous racing: Joint lookahead and steering-gain control with PPO," arXiv:2602.18386, 2026.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[6] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, 2011, pp. 627–635.

[7] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, "Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing," in *Proc. American Control Conf.*, 2007, pp. 2296–2301.

[8] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.
[9] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *Proc. IEEE Intelligent Vehicles Symp.*, 2015, pp. 1094–1099.
[10] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.
[11] J. Betz *et al.*, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open J. Intell. Transp. Syst.*, vol. 3, pp. 458–488, 2022.
[12] B. R. Kiran *et al.*, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, 2022.
[13] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
[14] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.
[15] M. Andrychowicz *et al.*, "What matters in on-policy reinforcement learning? A large-scale empirical study," in *Proc. ICLR*, 2021.
[16] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018, pp. 1861–1870.
[17] B. D. Evans, H. W. Jordaan, and H. A. Engelbrecht, "Comparing deep reinforcement learning architectures for autonomous racing," *Machine Learning with Applications*, vol. 14, p. 100496, 2023.
[18] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems 1*, 1989, pp. 305–313.
[19] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.
[20] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Commun. ACM*, vol. 27, no. 3, pp. 236–239, 1984.
[21] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
[22] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.